

- Debuggers
- Code coverage tools
- Static code checking tools
- Dynamic code checking tools
- Profiling tools
- Code formatters
- Documentation generators
- Open source libraries

Integrated Development Environments

An Integrated Development Environment (IDE) is an application that provides comprehensive facilities for software development. There are a large number of IDEs available for C programmers. Some of the popular open source IDEs include Code::Blocks, CodeLite, Geany, etc. In this race, Code::Blocks is the clear winner because of its ability to perform debugging, profiling, code completion, code coverage testing and static code analysis. Moreover, if you are an expert user of Code::Blocks, there won't be many practical situations for which you will need another tool for program analysis. Code::Blocks is a cross-platform application that will work on Windows, Linux and Mac OS X. The different compilers supported include GCC, MinGW, Digital Mars, Microsoft Visual C++, Borland C++, LLVM Clang, etc. The latest stable version is Code::Blocks 13.12, which was released on December 27, 2013.

Build automation tools

Build automation is the process of automating tasks like compiling source code to binary code, running tests and deploying the final application. Here too, we have a wide variety of tools to select from. Some of the popular tools include Make, Waf, and Automake. Waf is a Python-based tool which is gaining popularity. But Make is still the preferred tool for C developers. C program compilation can be automated with the Make utility by executing the command *make*. This will execute the Makefile, which should contain all the rules to automatically compile and build an executable from the source code. The rules mention prerequisite files and actions to be carried out in order to obtain a target file. You can download the source code of the C program and a Makefile from http://opensourceforu.com/article_source_code/sept14/make.zip. The zip file contains two C program files *main.c* and *fun.c*, a Makefile to build the source files and an image showing the output of Make.

There is some criticism about the Make utility regarding the rigid structure and syntax of the Makefile. Some criticism is based on the fact that Make offers too little in comparison with other tools like Waf. But, personally, I believe the best feature of Make is its simplicity and the criticism arises from the fact that it has been in use for the past 30 years. As often quoted, familiarity breeds contempt and with that, I rest my case.

Another tool which is often mentioned in conjunction with build automation is Automake, which is part of the GNU toolchain. Automake is an autotool used to automatically generate Makefiles for the Make utility.

Revision control systems

Another important tool that should be in the arsenal of a C programmer is a revision control system, a.k.a. a version control system. A revision control system is used when the changes made to the software being developed are very high, which makes it difficult to version properly. Here too, we have to choose from a wide list of tools, which include Mercurial, RCS, CVS, etc. Among all the different tools available, Mercurial is very simple and efficient.

The command *hg init* is used to initialise the repository by creating a directory called *.hg*. The *hg status* command tells us the status of the files being tracked by the repository. *hg add file_name* is used to add files for tracking. You can use the command *hg commit* to save different versions of your program files. *hg log* gives the details of different versions being tracked by the repository. And to revert to older versions, use *hg revert -r version_number*.

One common error message that haunts first time users of Mercurial is '*abort: no username supplied (see 'hg help config')*'. If you get this error, you should edit the file *.hgrc* located inside your home directory and add the following lines of text:

```
[ui]
username = Your Name <your@email.com>
```

Debuggers

For novices, writing a program devoid of syntax errors is a difficult task. But the real difficulty occurs while dealing with run time errors. As the size of the program increases, identifying bugs that cause run time errors becomes more and more difficult. In order to debug large programs, we ought to get help from debuggers like GDB, DBX, DDD, etc. The debugger GDB (GNU Debugger) has become so popular and influential that the selection of a debugger is made very easy for most of the programmers. In fact, most other debuggers are influenced by GDB, with DBX being the only real competitor that is not based on the philosophy of GDB. The success of GDB even led to the development of a GUI-based debugger called DDD (Data Display Debugger) based on GDB.

GDB allows you to inspect various program parameters during and after program execution. Errors like segmentation faults are identified with relative ease using GDB. The C program fragment given below contains a run time error and GDB is used to uncover it.

```
int a=111,b=0,div;
div=a/b;
printf("After division result is %d\n",div);
```

The program compiles without any syntax errors but contains a logical error. The program should be compiled as follows to enable debugging with GDB: *gcc -g program_name.c*. The option *-g* enables built-in debugging support required by GDB. You can start the GDB debugger by using